

ПРАКТИЧЕСКАЯ РАБОТА №4

МАНИПУЛИРОВАНИЕ ИЗОБРАЖЕНИЕМ

Цель работы: научиться на Python с использованием внешней библиотеки PIL копировать изображение, изменять размеры изображения, вырезать область на изображении и изменять размеры вырезанной области, вращать изображение.

Подготовка к выполнению заданий

1. Поместите в текущую папку набор цветных изображений и изображений в оттенках серого.
2. Поместите не в текущую папку набор других цветных изображений и изображений в оттенках серого.
3. На компьютере должен быть выход в интернет.
4. Установите внешнюю библиотеку PIL.

Задание 1. Копирование и изменение размера изображения

1. Скопируйте открытое изображение в и выведите на экран полученную копию

Подключаем модуль:

```
from PIL import Image
```

Открываем файл:

```
img = Image.open("foto.jpg")
```

Создаем копию:

```
img2 = img.copy()
```

Просматриваем копию:

```
img2.show()
```

2. Создайте уменьшенную версию изображения указанного размера.

Сначала откройте некоторое изображение и выведите его размеры

```
img=Image.open("foto.jpg")
```

```
img.size # Исходные размеры изображения
```

В итоге в консоль выведется размер этого изображения:

```
(800, 600)
```

Измените размер изображения пропорционально с помощью метода

```
thumbnail (<Размер> [, <Фильтр>]) .
```

Обратите внимание на то, что изменение размера производится пропорционально, иными словами, за основу берется минимальное значение, а второе значение вычисляется пропорционально первому.

Метод изменяет само изображение и ничего не возвращает.

Фильтры **NEAREST**, **BILINEAR**, **BICUBIC** или **ANTIALIAS** .

```
img.thumbnail((400, 300), Image.ANTIALIAS)
```

```
img.size # Изменяется само изображение
```

В итоге в консоль выведется размер измененного изображения:

```
(400, 300)
```

```
img = Image.open("foto.jpg")
```

```
img.thumbnail((400, 100), Image.ANTIALIAS)
```

```
img.size # Вывод размера изображения
```

Увидим, что размер изменяется пропорционально

В итоге в консоль выведется размер измененного изображения:

```
(133, 100)
```

3. Создайте версию изображения указанного размера

Сначала откройте некоторое изображение и выведите его размеры

```
img = Image.open("foto.jpg")
```

```
img.size # Исходные размеры изображения
```

В итоге в консоль выведется размер измененного изображения:

```
(800, 600)
```

Теперь изменим само изображение

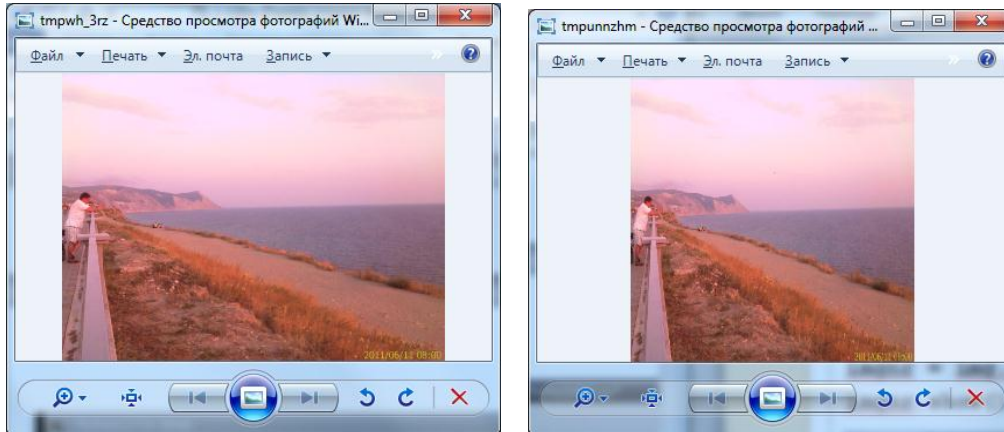
```
img.show()
```

```
newsize = (400, 400)
```

```
imgnr = img.resize(newsize)
```

`imgnr.show()` # Изменяется само изображение

В итоге получим:



Задание 2. Вырезка области изображения и изменение его размеров

1. Вырежьте прямоугольный фрагмент из исходного изображения.

```
>>> img = Image.open("foto.jpg")
>>> img2 = img.crop( [0, 0, 100, 100] ) # Помечаем фрагмент
>>> img2.load() # Считываем фрагмент, создавая новое изображение
>>> img2.size
(100, 100)
```

2. Измените размеры области изображения

Сначала откройте некоторое изображение и выведите его рамеры

```
img = Image.open("tsveti.jpg")
img.size # вывод размеров изображения
img.show()
```

В итоге откроется исходное изображение и в консоль выведется размер исходного изображения:

```
(668, 438)
```

Затем возьмем кусок изображения

```
box = (100, 100, 300, 300) # берем кусок изображения:
```

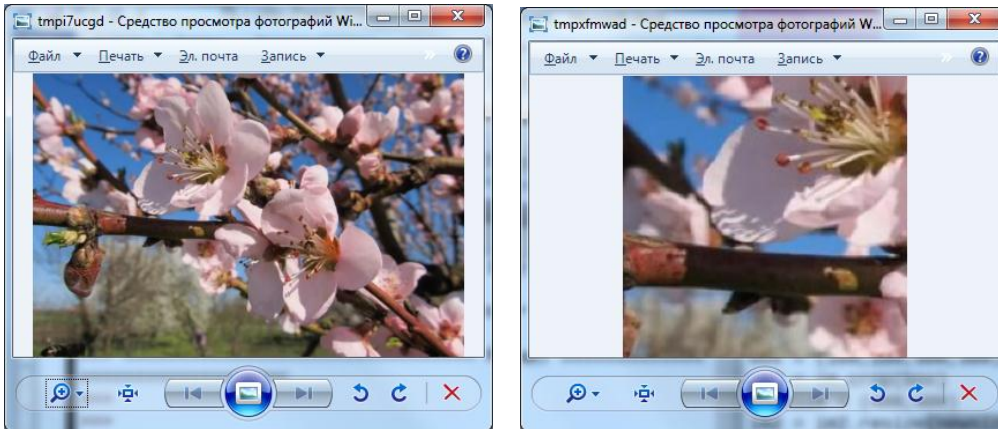
Затем изменяем само изображение

```
img2 = img.crop(box)
newsize = (400, 400)
```

```
img2nr = img2.resize(newsize)
```

```
img2nr.show() # Изменяется само изображение
```

Получим в итоге:



Задание 3. Вращение изображения

1. Осуществите вращение изображения. Но сначала откройте некоторое изображение и выведите его размеры в консоль.

```
img = Image.open("foto.jpg")
```

```
img.size # Исходные размеры изображения
```

В консоль выведется размер исходного изображения:

```
(800, 600)
```

Затем осуществите поворот на 90 градусов

```
img2 = img.rotate(90) # Поворот на 90 градусов
```

```
img2.size
```

В консоль выведется размер повернутого на 90 градусов изображения:

```
(600, 800)
```

```
img3 = img.rotate(45, Image.NEAREST)
```

```
img3.size # Размеры сохранены, изображение обрезано
```

```
(800, 600)
```

```
img4 = img.rotate(45, expand=True)
```

```
img4.size # Размеры увеличены, изображение полное
```

```
(991, 990)
```

2. Получите горизонтальный и зеркальный образ изображения:

```
>>> img2 = img.transpose(Image.FLIP_LEFT_RIGHT)
>>> img2.show() # Горизонтальный зеркальный образ
>>> img3 = img.transpose(Image.FLIP_TOP_BOTTOM)
>>> img3.show() # Вертикальный зеркальный образ
```

и следующими способами:

```
>>> img4 = img.transpose(Image.ROTATE_90)
>>> img4.show() # Поворот на 90° против часовой стрелки
>>> img5 = img.transpose(Image.ROTATE_180)
>>> img5.show() # Поворот на 180°
```

ЛИТЕРАТУРА К РАБОТЕ

1. Python Imaging Library (Fork) . – URL: <https://pypi.org/project/Pillow/> (дата обращения: 10.05.2023).
2. Прохоренок, Н. А. Python 3. Самое необходимое: Пособие / Прохоренок Н.А., Дронов В.А. - СПб:БХВ-Петербург, 2016. - 464 с. ISBN 978-5-9775-3631-8.-Текст:электронный).–URL: https://codernet.ru/books/python/python_3_samoe_neobxodimoe_proxorenok/ (дата обращения: 10.05.2023).
3. Python 3 и PyQt 6. Разработка приложений/ Н. А. Прохоренок, В. А. Дронов. -СПб.: Б:ХВ-Петербург, 2023.- 832 с.: ил.-(Профессиональное программирование). – URL: <https://coollib.net/b/617121-vladimir-dronov-python-3-i-pyqt-6-razrabotka-prilozheniy/readp?p=186&cnt=9000> (дата обращения: 10.05.2023).